

Operating System复习

简介

操作系统的五大功能：处理机管理、存储器管理、设备管理、文件管理、作业管理

操作系统的特征：并发、共享、虚拟、异步

虚拟存储管理系统的基础是程序的==局部性理论==

==系统调用==

中断与陷入的主要区别：陷入通常由处理机正在执行的现行指令引起，而中断则是由与现行指令无关的中断源引起的

原语：处于操作系统底层、不可中断执行的操作

系统调用：操作系统提供给用户程序与操作系统内核之间的接口，通过系统调用将用户级的请求传递到内核态，并在内核态调用相应的函数实现相应的功能

1. 用户将系统调用号和所需参数压入堆栈，执行一个陷入指令，将CPU状态从用户态转为内核态，同时保护中断现场
2. 分析系统调用类型，转入相应的系统调用子程序
3. 在系统调用子程序执行结束后，恢复中断现场，返回被中断的进程或新进程并继续执行

进程与线程

竞态：在并发系统中，当多个进程或线程同时访问共享资源，访问结果依赖于访问时序

进程

进程是系统进行资源分配和调度的一个独立单位

进程控制块(Process Control Block, PCB)：由程序段、数据段和PCB构成了进程实体

进程的5个状态：运行态、就绪态、阻塞态、（创建态、中止态）

进程的创建

1. 申请空白PCB，分配一个唯一的进程标识号
2. 为新进程分配其运行所需的资源
3. 初始化进程控制块PCB
4. 如果进程就绪队列能够接纳新进程，便将进程==插入就绪队列==

进程的通信

- **共享存储**：对共享空间的读写实现信息交换
- **消息传递**：通过发送消息和接受消息两个原语进行数据交换
- **管道通信**：管道是一种特殊的共享文件，数据在管道中先进先出，允许两个进程按照生产者-消费者方式进行通信

线程

线程是进程中的一个实体，是被系统独立调度和分派的最小单位，自己不拥有系统资源

运行态、就绪态、阻塞态

进程与线程的区别

- 相同点：都是执行单位（都可以被 CPU 调度执行）、都有生命周期、都可并发执行
- 不同点：拥有资源、切换开销、资源共享

线程的实现方式

- 用户级线程：内核意识不到线程的存在，调度仍然以进程为单位
- 内核级线程：同一进程中的线程切换需要从用户态转到核心态
- 组合方式：包含用户级线程和内核级线程

CPU调度

三级调度

- 高级调度（作业调度）：根据某种算法，决定将外存上处于后备队列的那几个作业调入内存，为他们创建进程、分配必要的资源，并将它们放入就绪队列
- 中级调度（内存调度）：主要功能是提高内存利用率和吞吐量，将暂时不能运行的进程调至外存等待，把外存上已具备运行条件的进程再重新调入内存
- 低级调度（进程调度）：根据某种算法，决定就绪队列中的那几个进程应获得处理机，并由分派程序将处理机分配给被选中的进程

调度程序

排队器、分派器、上下文切换器

进程调度方式

- 非抢占调度方式：一旦一个进程开始执行，它会一直运行直到自愿放弃CPU
- 抢占调度方式：操作系统可以在任何时候中断当前正在运行的进程，并选择另一个进程来执行

调度算法

先来先服务（FCFS, First-Come, First-Served）

- 按照进程到达的顺序依次执行，非抢占式

短作业优先（SJF, Shortest Job First）

- 根据预测的下一个CPU爆发时间选择最短的进程执行

高响应比优先（Highest Response Ratio Next, HRRN）

- 响应比 = (等待时间+预计运行时间) / 预计运行时间

轮转法（Round Robin, RR）

- 每个进程轮流获得一个固定长度的时间片（time slice）来运行，抢占式调度

多级反馈队列调度（Multilevel Feedback Queue Scheduling）

- 使用多个就绪队列，每个队列有自己的优先级和时间片大小，进程可以根据其行为（如是否超时）在不同队列之间移动

同步与互斥

生产者-消费者问题

一组生产者进程和一组消费者进程共享一个有限缓冲区，生产者负责生成数据并将其放入缓冲区，而消费者则从缓冲区取出数据进行处理

定义如下三个信号量：

- `empty`：表示空闲缓冲槽数量，初始值等于缓冲区的最大容量。
- `full`：表示已填充缓冲槽数量，初始值为0。
- `mutex`：用于保护对缓冲区的访问，确保同一时间只有一个线程能修改缓冲区内容，初始值为1（二进制信号量）。

```
semaphore empty = 1;
semaphore full = 1;
semaphore mutex = 1;

// 生产者
while (true) {
    produce_item(item);
    wait(empty);      // 等待有空位
    wait(mutex);      // 进入临界区
    insert_item(buffer, item);
    signal(mutex);    // 离开临界区
    signal(full);     // 增加已填槽数
}

// 消费者
while (true) {
    wait(full);       // 等待有物品
    wait(mutex);      // 进入临界区
    remove_item(buffer, item);
    signal(mutex);    // 离开临界区
    consume_item(item);
    signal(empty);    // 增加空槽数
}
```

读者-写者问题

一组读者进程和一组写者进程共享一个资源或数据结构，有两类进程：

1. 读者（Reader）：只读取共享资源的数据，不会对其进行修改。
2. 写者（Writer）：可以读取并修改共享资源的数据。

写者优先（一旦有写者等待，系统就会优先处理写者的请求）的伪代码：

- `mutex`：用于保护对读者计数器 `rc` 和其他变量的访问，初始值为1。
- `db`：如上所述，控制写者访问。
- `wr`：用于指示是否有写者正在等待，初始值为1。

```

int rc = 0; // 记录当前有多少读者正在读取共享资源
semaphore mutex = 1; // 保护对rc的更新操作
semaphore db = 1; // 控制对共享资源的访问权限
semaphore wr = 1; // 保证写者优先,如果有写者在等待,则新的读者不能开始读取

// 读者伪代码
reader() {
    wait(wr); // 确保没有写者在等待
    wait(mutex);
    rc++;
    if (rc == 1) {
        wait(db); // 如果是第一个读者,则锁定写者
    }
    signal(mutex);
    signal(wr);

    // 读取共享资源
    read();

    wait(mutex);
    rc--;
    if (rc == 0) {
        signal(db); // 如果没有读者了,则解锁写者
    }
    signal(mutex);
}

// 写者伪代码
writer() {
    wait(wr); // 阻止更多读者开始
    wait(db); // 锁定所有读者和写者
    // 修改共享资源
    write();
    signal(db);
    signal(wr);
}

```

哲学家进餐问题

五个哲学家围坐在一张圆形桌子旁的情景,每个哲学家之间有一根筷子,总共五根。哲学家们交替进行思考和吃饭两种活动,当一个哲学家饥饿时,他必须拿起左右两边的两根筷子才能开始吃饭;吃完后,他会放下筷子继续思考

- 每个哲学家可以处于三种状态之一——思考 (thinking)、饥饿 (hungry) 或者吃饭 (eating)
- 一个哲学家只有在拿到左右两边的两根筷子后才能开始吃饭
- 吃完饭后,哲学家会放下手中的筷子并继续思考
- 如果一个哲学家拿起了第一根筷子但无法拿到第二根,那么他应该放回第一根筷子并等待一段时间后再尝试

采用“只有当左右两边的筷子都可用时才允许抓起筷子”的策略:

```

semaphore chopstick[5] = {1,1,1,1,1};
semaphore mutex = 1;

```

```
Pi{
    while(true){
        wait(mutex);
        wait(chopstick[i]);
        wait(chopstick[(i+1)%5]);
        signal(mutex);
        eat;
        signal(chopstick[i]);
        signal(chopstick[(i+1)%5]);
        think;
    }
}
```

死锁

多个进程竞争资源，互相等待对方手里的资源，使得各个进程都被阻塞

产生死锁的必要条件

- **互斥条件**：进程对所分配到的资源进行排他性使用
- **不可剥夺条件**：进程已获得的资源在未使用完之前不能被抢占，只能在进程使用完时释放
- **请求和保持条件**：进程已经保持了至少一个资源，但又提出了新的资源请求，该资源已被其他进程抢占
- **循环等待条件**：在发生死锁时，必然存在一个进程—资源的循环链

预防死锁

- 破坏互斥条件，将只能互斥使用的资源改为允许共享使用
- 破坏不可剥夺条件，当一个已保持某些不可抢占资源的进程，提出新的自愿请求而得不到满足时，它必须释放已保持的所有资源
- 破坏请求和保持条件
 - 第一种协议，所有进程在开始运行之前，必须一次性申请其在整个运行过程中所需要全部资源
 - 第二种协议，允许一个进程只获得运行初期的所需资源后，便开始运行，进程运行过程中再逐步释放已分配给自己的、且已用完的全部资源，然后再请求新的所需资源
- 破坏循环等待条件，对系统所有资源类型进行线性排序并赋予不同编号，每个进程必须按序号递增的顺序请求资源，若一个已经请求道一些高序号资源的进程又想请求低序号的资源，必须先释放所具有相同和更高序号的资源后才能申请

银行家算法

1. 安全状态：如果存在一种或多种序列，使得按照该序列依次运行每个进程时，它们都能获得所需的资源并最终释放所有已分配的资源，那么这个状态称为“安全状态”。否则，称为“不安全状态”。
2. 资源分配决策：每当一个进程请求资源时，银行家算法会检查此次分配是否会将系统带入不安全状态。如果是，则拒绝此次请求；如果不是，则允许分配资源。
3. 预防死锁：通过确保每次资源分配都保持系统在安全状态下，银行家算法有效地预防了死锁的发生。

为了实现银行家算法，需要维护以下几个数据结构：

- Max[i]：表示进程 **i** 可能请求的最大资源数量。
- Allocation[i]：表示已经分配给进程 **i** 的资源数量。

- $Need[i] = Max[i] - Allocation[i]$: 表示进程 i 还需要多少资源才能完成任务。
- $Available[]$: 表示当前系统中可用的资源总数。

内存

分区存储管理方案

将内存划分为若干个==连续的分區==，每个分区可以容纳一个作业

每个分区与作业——对应，分区大小可能与作业需求不完全匹配

1. **固定分区分配**: 事先将内存划分为若干大小固定的分区，每个分区装入一个作业（进程），==分区大小可以相同也可以不同==，优点是简单易行，缺点是容易产生==内部碎片==，且分区大小限制了可装入作业的大小。
2. **动态分区分配**: 根据进程大小动态地划分内存分区，分配时找合适空闲分区分配给进程，会产生==外部碎片==，解决外部碎片可采用紧凑技术（移动进程，使空闲分区合并），常用的分配算法见下方。
3. **可重定位分区分配**: 在动态分区基础上，为解决外部碎片问题，通过动态重定位技术（借助重定位寄存器），移动进程在内存中的位置，合并空闲分区，提高内存利用率。

四种动态分区算法

- 首次适应算法FF，每次从链首开始顺序查找，知道找到一个大小能满足要求的空闲分区位置
- 邻近适应算法NF，每次从上次找到的空闲分区的下一个空闲分区开始查找，知道找到一个大小能满足要求的空闲分区
- 最佳适应算法BF，每次查找时，总是把能满足要求又是最小的空闲分区分配给作业
- 最坏适应算法WF，每次查找时，总是挑一个最大的空闲区

页式存储管理方案

页 (Page)

- 逻辑地址空间被划分为固定大小的页（如4KB、8KB等）。
- 每一页是逻辑上的一个存储单元。

页框 (Frame)

- 物理内存被划分为与页大小相等的页框。
- 每一页框是物理内存上的一个存储单元。

页表 (Page Table)

- 用于记录逻辑地址空间的页和物理内存的页框之间的映射关系。
- 每个作业对应一个页表。

逻辑地址由页号和页内偏移量组成

一个作业的各页可以分散存储在内存的不同位置

用于解决**内存碎片问题**，但不体现程序的逻辑结构

1. **基本原理**：将内存空间和程序空间都划分为固定大小的页（如每页 4KB），程序的逻辑地址分为页号和页内偏移量，通过页表（页号映射到页框号）实现逻辑地址到物理地址的转换，CPU 访问内存时先查页表找到物理块号，再结合页内偏移量确定物理地址。
2. **页表结构及优化**：页表可能占用较大内存空间，可采用多级页表（如二级页表、三级页表等，将页表再分页管理）、快表（TLB，存放近期常用页表项，是一种高速缓存，加快地址转换速度）等方式优化。

两级页表

[3.1.9 OS之二级页表的原理和地址结构 二级页表地址格式-CSDN博客](#)

段式存储管理方案

段的大小不固定，由程序的逻辑需求决定

地址由段号和段内偏移量组成，段号表明逻辑含义

更贴近程序逻辑结构，但可能会产生**外部碎片**

1. **基本原理**：按程序的逻辑结构（如代码段、数据段、堆栈段等）划分成不同的段，每段有自己的段名和长度，逻辑地址由段号和段内偏移量组成，通过段表（记录段号对应的内存起始地址、段长等信息）实现地址转换，便于程序和数据的模块化组织、共享和保护。
2. **优缺点**：优点是符合程序逻辑结构，便于程序段的动态增长、共享和保护；缺点是段的长度不一，内存分配管理相对复杂，容易产生外部碎片。

段页式存储管理方案

结合了段式和页式的优点，先按段式划分程序，每个段再按页式划分，逻辑地址由段号、段内页号、页内偏移量组成，访问内存时先通过段表找到对应段的页表，再通过页表找到物理块，进行地址转换，这种方式管理复杂但综合性能较好。

虚拟存储管理

1. **基本概念**：基于**局部性原理**，在程序运行时，不必将整个程序和数据都放入内存，只把当前需要的部分调入内存，其余放在外存（如磁盘），当需要访问外存部分时再调入内存，从用户角度感觉内存空间得到扩充。

2. 页面置换算法

当内存空间已满，需要调入新页面时，决定置换出哪个已在内存的页面，常见算法有：

- **先进先出 (FIFO)**：按页面进入内存的先后顺序置换，容易实现，但可能出现**Belady 异常**（增加物理内存页面数，缺页次数反而增加），性能不佳。
- **最近最久未使用 (LRU)**：置换最近最长时间未被使用的页面，符合程序局部性规律，缺页率相对较低，但实现成本较高，需记录页面使用时间顺序信息。
- **最佳置换算法 (OPT)**：理论上选择以后永远不会使用或在最长时间不会再使用的页面进行置换，缺页率最低，但无法预知未来页面使用情况，实际中难以实现，常作性能对比参照。

3. **页面分配策略**：决定为每个进程分配多少物理页面，有固定分配局部置换（给进程固定页面数，缺页时只在分配的页面内置换）、可变分配全局置换（根据系统空闲页面情况给进程分配页面，缺页时可从全局空闲页面中置换）、可变分配局部置换（初始分配一定页面数，缺页时可适当增加分配页面数，且只在自身页面内置换）等策略。

抖动

虚拟存储管理系统中，频繁页面置换使系统效率剧降的现象，系统像在内存和磁盘间来回折腾页面，执行程序任务的时间变少。

抖动产生的主要原因：

- 内存资源不足
- 页面置换算法不合理
- 程序的局部性差

文件管理

文件是具有符号名的、在逻辑上具有完整意义的一组相关信息项的有序序列，是计算机存储信息的基本单位，如文档、程序、图像等都以文件形式存在。

文件控制块（File Control Block, FCB）：存放控制文件需要的各种信息的数据结构，FCB的有序集合称为文件目录

inode：**inode** 是一种数据结构，存储与文件相关的元数据（metadata），但不包含文件的实际内容或文件名

文件打开后，通过==文件描述符==来操作文件

文件的物理结构

1. **连续分配**：目录中包含文件第一个磁盘块的块号和所占用的块数，支持顺序访问和直接访问
2. **链接分配（隐式）**：通过链表的方式存储文件，每个文件的磁盘块包含数据和指向下一个磁盘块的指针，无法直接访问指定位置的数据
3. **链接分配（显式）**：将链接各物理块的指针显式存放在一张链接表中，称为文件分配表（File Allocation Table, FAT），每个表项中存放指向下一个盘块的指针，支持顺序和直接访问，由于表放在内存中，检索速度快
4. **索引分配（单级/多级）**：为每个文件分配一个索引块，索引块记录了该文件所有数据块的地址
5. **混合索引分配**：分为直接块、一级间址、二级间址、三级间址（==一个逻辑块可以指向多个物理块，突破传统 inode 数据块指针的限制，大幅提高文件系统的存储容量==）

文件的目录结构

1. **单级目录结构**：最简单的目录结构，所有文件都在同一个目录下，优点是简单，缺点是文件数量多时查找困难，且文件名不能重复，不便于文件管理和共享。
2. **两级目录结构**：分为主目录和用户目录，不同用户有自己的子目录，可在一定程度上解决文件名冲突问题，方便用户管理自己的文件，但仍不够灵活，不能很好地反映文件的层次关系。

3. **树形目录结构**：最常用的目录结构，类似倒立的树，根目录下有多个子目录，子目录下又可细分，层次分明，便于文件分类管理、查找和共享，用户通过完整路径名（如“C:\Program Files\Microsoft Office\Word.exe”）定位文件，操作系统依据目录项（包含文件名、属性、存储位置等信息）管理文件。
4. **图形目录结构**：以图形化方式展示目录和文件关系，更直观，便于用户操作，常用于图形用户界面操作系统中，用户通过鼠标点击等操作浏览、管理文件。

文件系统

一个文件系统通常划分为以下几个主要区域：主引导记录（Master Boot Record, MBR）、分区表、多个磁盘分区

磁盘分区的组成：引导块、超级块、空闲块信息、i节点、根目录、文件和目录

文件存储空间管理

1. **空闲表法**：用一张表格记录磁盘上各个空闲块的==起始地址和长度==等信息，分配空间时查找合适空闲块，回收时更新表格内容，适用于连续分配方式，管理简单但表格维护开销较大。
2. **空闲链表法**：将磁盘空闲块通过链表形式连接，每个空闲块包含指向下一个空闲块的指针，分配时顺着链表查找合适空闲块，回收时将空闲块插入链表合适位置，灵活性高但查找效率相对较低。
3. **位示图法**：用每一位对应磁盘上的一个物理块，0 表示空闲，1 表示已占用，通过位运算进行空间分配和回收操作，占用空间小且便于快速查找空闲块，但需要额外的位运算操作。
4. ==**成组链接法**==：[成组链接法详解-CSDN博客](#)
 1. 磁盘中的空闲块被分成若干组，每组包含一部分空闲块的地址。
 2. 每组的第一个空闲块充当目录块，用于存储本组中其他空闲块的地址以及下一组目录块的地址。
 3. 第一组的空闲盘块总数和空闲盘块号存放在内存的专用栈中，称为==空闲盘块号栈==（包含空闲盘块号栈指针）。

*文件共享与文件保护

1. **文件共享**：允许多个用户或进程共享同一个文件，可通过硬链接（多个文件名指向同一个文件的索引节点，删除一个文件名不影响文件本身，除非所有硬链接都删除）和软链接（也叫符号链接，是一个特殊文件，指向另一个文件的路径，访问时通过路径查找目标文件）等方式实现，节省存储空间，提高资源利用率。
2. **文件保护**：防止文件被非法访问、篡改或破坏，常用方法有：
 - **设置访问权限**：如规定用户对文件的读、写、执行权限，通过文件属性或访问控制列表（ACL）来体现，不同用户或用户组有不同权限。
 - **加密**：对文件内容进行加密处理，只有拥有正确密钥的用户才能解密并访问文件，安全性高，但加密和解密会增加系统开销。

I/O

I/O 控制方式

1. **程序直接控制方式**：最原始的控制方式，CPU 不断循环查询设备的状态寄存器，判断设备是否准备好进行数据传输。若设备未准备好，CPU 就一直等待；若准备好了，CPU 则进行数据传输操作。这种方式 CPU 利用率极低，因为在等待设备准备期间，CPU 一直处于空闲状态。
2. **中断驱动方式**：当设备完成数据准备或数据传输结束等情况时，会向 CPU 发送中断请求信号，CPU 响应中断后暂停当前正在执行的程序，转而去执行相应的中断处理程序来完成数据传输等操作，之后再返回原程序继续执行。这种方式使得 CPU 在设备进行数据准备期间可以执行其他任务，提高了 CPU 的利用率。
3. **DMA方式**：在内存和设备之间开辟了一条直接的数据传输通路，==设备可以不经过 CPU 直接与内存进行批量的数据传输，只需在传输开始和结束时向 CPU 发送请求和通知==。
4. **通道控制方式**：==通道是一种具有特殊功能的处理器，它可以独立执行通道程序来控制设备与内存之间的数据传输，进一步减轻 CPU 的负担==，且能同时管理多台同类型或不同类型的设备，实现设备的并行操作。

缓冲

1. **缓冲的作用**：由于 CPU 和 I/O 设备在处理速度上存在很大差异，设置缓冲区可以缓和这种速度不匹配的矛盾，提高 CPU 和 I/O 设备的并行性，同时减少数据传输过程中的中断次数等。例如，在网络数据接收时，网络接口的数据传输速度可能不稳定且相对较慢，而 CPU 处理数据速度很快，通过缓冲区先暂存网络传来的数据，让 CPU 可以在合适时间批量处理，避免频繁等待。
2. **缓冲的类型**：
 - **单缓冲**：在设备和 CPU 之间设置一个缓冲区，设备先将数据写入缓冲区，然后 CPU 再从缓冲区读取数据进行处理。它结构简单，但对于连续快速的数据传输场景，效率提升有限，常用于简单的、数据量不大且对传输速度要求不高的情况，比如键盘输入数据的临时存储。
 - **双缓冲**：设置两个缓冲区，设备向一个缓冲区写入数据的同时，CPU 可以从另一个缓冲区读取数据，两个缓冲区交替使用，使得设备和 CPU 能更好地并行工作，提高了数据传输效率，常用于图像显示等需要一定并行处理能力的场景，一边可以接收新的图像数据到一个缓冲区，另一边 CPU 可以对另一个缓冲区中的图像数据进行渲染等操作。
 - **循环缓冲**：由多个缓冲区组成一个循环队列的形式，适用于高速设备的数据传输，比如磁盘读写操作，多个缓冲区可以连续不断地接收和处理数据，保证数据的流畅传输，防止因设备和 CPU 速度差异导致的数据丢失或处理延迟等问题。

SPOOLing

==将一台物理I/O设备虚拟为多台逻辑I/O设备，允许多个用户共享一台物理I/O设备==

- 输入井：在磁盘上开辟的存储区域，模拟脱机输入时的磁带，用于收容I/O设备输入的数据
- 输出井：在磁盘上开辟的存储区域，模拟脱机输出时的磁带，用于收容用户进程输出的数据
- 输入进程：模拟脱机输入时的外围控制机
- 输出进程：模拟脱机输出时的外围控制机

输入 SPOOLing：

- 由输入进程控制，外设将数据送入输入缓冲区（内存），再存放到输入井
- CPU直接从输入井中读取数据到内存

输出 SPOOLing：

- 由输出进程控制，输出数据由内存送入输出井，待输出设备空闲时将数据经输出缓冲区（内存）输出至输出设备

系统对于用户的打印输出，但并不真正把打印机分配给该用户进程，而是先在输出井中申请一个空闲盘块区，并将要打印的数据送入其中；然后为用户申请并填写请求打印表，将该表挂到请求打印队列上。若打印机空闲，输出程序从请求打印队首取表，将要打印的数据从输出井传送到内存缓冲区，再进行打印，直到打印队列为空。

*设备分配

1. **设备分配原则**：根据设备的类型、数量以及各进程的需求，按照一定的策略来合理分配设备资源。对于独占设备（如打印机，同一时间只能被一个进程使用），通常采用独占分配方式，确保使用的独占性；对于共享设备（如磁盘，多个进程可同时访问不同区域），可以多个进程同时使用，通过合适的调度算法来协调访问顺序，避免冲突。同时还要考虑设备的使用效率、公平性以及避免死锁等问题。
2. **设备分配算法**：常见的有先来先服务算法（按照进程申请设备的先后顺序进行分配，简单公平，但可能对急需设备的重要进程不利）、优先级高的先分配算法（根据进程的优先级高低来分配设备，优先满足重要的、优先级高的进程需求，但要注意避免低优先级进程长期得不到设备的“饥饿”现象）等。

*I/O 软件系统层次模型

1. **用户层软件**：这是最靠近用户的一层，通常由应用程序中的 I/O 相关函数等组成，例如在 C 语言中调用 `printf` 函数进行输出操作，它通过系统调用接口向下面的层次发起 I/O 请求，为用户提供了方便的操作接口来实现简单的 I/O 功能，并且可以根据应用程序的需求对数据进行一定的格式处理等。
2. **设备独立性软件**：也叫与设备无关的 I/O 软件，它主要负责实现逻辑设备名到物理设备名的转换，为用户层软件提供统一的 I/O 操作接口，而不管底层具体使用的是哪种实际设备。比如，应用程序只需用通用的“打印机”这个逻辑设备名来发起打印请求，设备独立性软件会根据系统配置等情况找到对应的实际物理打印机设备来执行操作，同时还处理设备的分配、缓冲管理、出错处理等功能，提高了 I/O 系统的通用性和可移植性。
3. **设备驱动程序**：它是直接与硬件设备打交道的软件，负责将操作系统上层的 I/O 指令转化为设备能够理解和执行的特定控制信号，驱动设备完成相应的操作，如控制磁盘读写头的移动、打印机的打印头动作等，并且将设备的状态信息（如设备是否故障、是否忙碌等）反馈给上层软件，不同类型的设备都有对应的专门驱动程序，需要根据设备的硬件特性进行编写和适配。
4. **中断处理程序**：当设备完成一次 I/O 操作或者出现异常情况时，会通过中断机制向 CPU 发送中断信号，中断处理程序会立即响应中断，暂停当前正在执行的程序，去处理与该中断相关的事宜，比如完成数据的后续传输、对设备故障进行相应处理等，处理完后再恢复原程序的执行，确保设备和 CPU 之间能高效协调工作。
5. **硬件**：就是实际的 I/O 设备本身以及与之相关的硬件电路、接口等，是整个 I/O 系统的物理基础，如磁盘的盘片、磁头、电机等部件，以及与计算机主板连接的接口线路等，它们执行具体的物理数据读写、信号转换等操作。

其他

结合GPL许可证评价一下自由软件运动

==GPL许可证是一种广泛使用的自由软件许可证，自由软件运动强调软件的自由性，即用户对软件拥有自由使用、修改和分发的权利。==

1. 打破了传统软件产业的垄断格局，使得更多的企业和个人能够参与到软件的开发和创新中来
2. 促进了企业间的竞争与合作，共同推动技术的发展和應用
3. 推动了“**服务型商业模式**”的发展，许多公司通过提供自由软件的支持服务、定制开发和培训等方式盈利
4. 为自由软件提供了法律保护，防止专有软件厂商将自由软件的代码私有化

中美冲突下的我国计算机操作系统发展策略

- 核心技术研发自主化，加大对操作系统核心组件的研发投入
- 软硬件协同优化，加强国产操作系统与国产芯片的适配，优化性能和兼容性
- 构建国产操作系统生态
- 政策支持与推广
- 教育和人才培养

计算机操作系统未来的发展方向

- 虚拟化
- 多核
- 更大的地址空间
- 安全性

编译内核的步骤

在内核目录下创建一个符号链接，将当前目录下的 `linux-6.10.10` 目录链接到 `/usr/src/linux`，让系统知道正在使用的内核源代码目录。

```
sudo ln -s `pwd`/linux-6.10.10 /usr/src/linux
```

切换目录，配置config（虽然没有修改config）

```
cd /usr/src/linux
cp /boot/config-`uname -r` .config
make oldconfig
```

在新内核中嵌入自己的学号（原先已经嵌入，也不用修改）：

```
make gconfig
```

清理旧的构建文件：

```
make clean
```

编译内核（这里使用8个并行任务）：

```
make -j8
```

安装模块：

```
sudo make modules_install
```

安装内核：

```
sudo make install
```

重启

```
sudo reboot
```